

Privilege Leakage and Information Stealing through the Android Task Mechanism

Megha Mathew

Rishindra Mateti

Department of Computer Science & Engineering

Wright State University

Dayton, OH 45435

Outline

- 1. Introduction**
- 2. Problem Statement**
- 3. Threat Model**
- 4. Attack Overview**
- 5. Attack Details**
- 6. Attack Performance**
- 7. Solution Overview**
- 8. Evaluation**
- 9. Related Work**
- 10. Conclusion**
- 11. Q&A**

Introduction

- Android dominates the smartphone market (82.8% share).
- Over 1.9 million apps on Google Play.
- Security relies on isolation and permissions.
- Problem: Android task mechanism facilitates app collaboration but can lead to privilege leakage and information stealing.

Problem Statement

Isolation vs. Collaboration:

- Android isolates apps for security but allows collaboration through tasks.
- Task mechanism can leak privileges, enabling malicious apps to steal sensitive information.

Threat Model:

- Malicious app (AppM) can interfere with benign app (AppB) without special permissions.

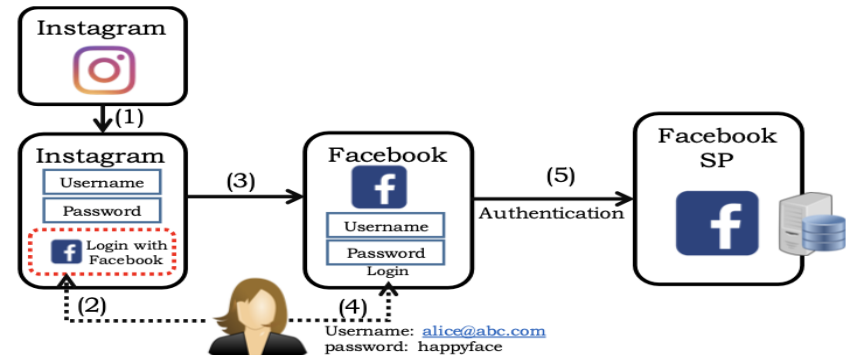
Android Task Mechanism

Key Components:

- Activities: Single screen with UI.
- Tasks: Group of activities from different apps working together.

Task Attributes: launchMode, taskAffinity, allowTaskReparenting.

Example: Instagram and Facebook collaboration for Single Sign-On (SSO).



(a) Single-Sign-On Architecture

Threat Model

Scenario:

- Two apps: Benign (AppB) and Malicious (AppM).
- AppM can join AppB's task and exploit privileges.

Assumptions:

- AppM requires no special permissions initially.
- AppM can later request permissions like INTERNET or READ_EXTERNAL_STORAGE.

Attack Overview

Four Proof-of-Concept Attacks:

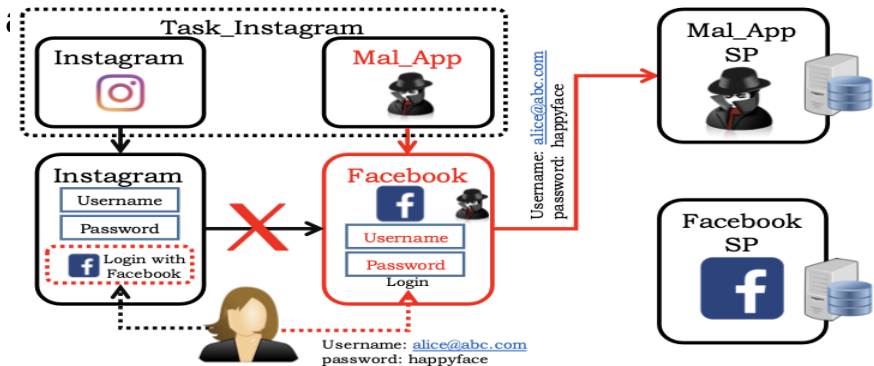
- UI Phishing
- Activity-in-the-Middle Attack
- Gallery Stealing
- Screenshot Capturing

Common Permissions
READ_EXTERNAL_STORAGE.

Required: INTERNET,

UI Phishing Attack

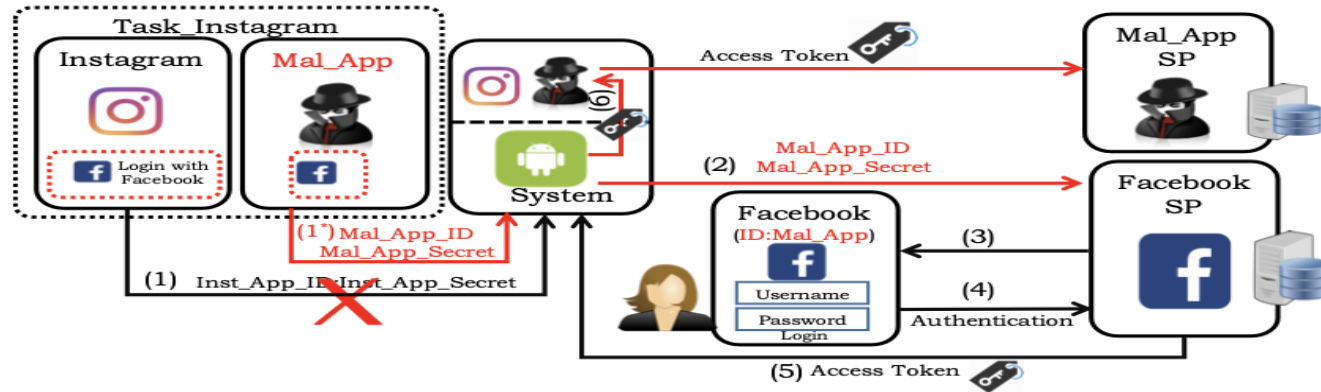
- Objective: Spoof login UI to steal credentials.
- Steps:
 1. AppM declares same taskAffinity as Instagram.
 2. AppM monitors Instagram's activity.
 3. AppM overrides Facebook login UI with :
- Visual: Diagram showing the attack flow.



(b) UI Phishing Attack

Activity-in-the-Middle Attack

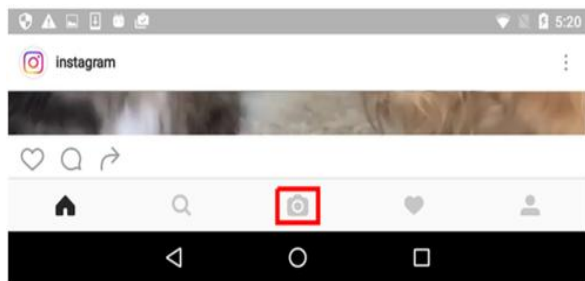
- Objective: Intercept OAuth 2.0 tokens during SSO.
- Steps:
 1. AppM intercepts Instagram's request to Facebook.
 2. AppM sends its own request to Facebook.
 3. AppM receives the access token instead of Instagram.



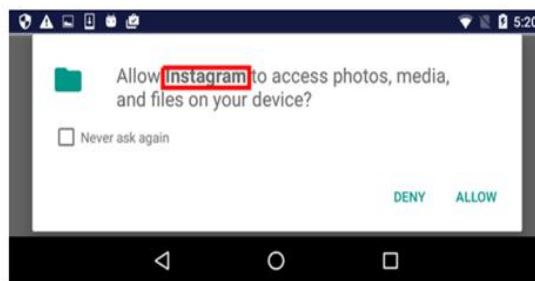
(b) Man-in-the-Middle Attack

Gallery Stealing Attack

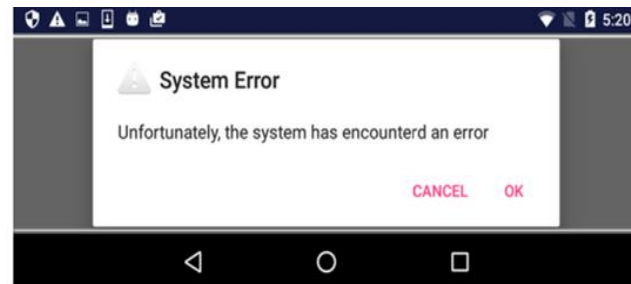
- Objective: Steal photos from the gallery.
- Steps:
 1. AppM monitors Instagram's camera activity.
 2. AppM kills Instagram and requests gallery access.
 3. AppM uses tapjacking to disguise the permission dialog.
- Visual: Screenshots of the attack in action.



(a)



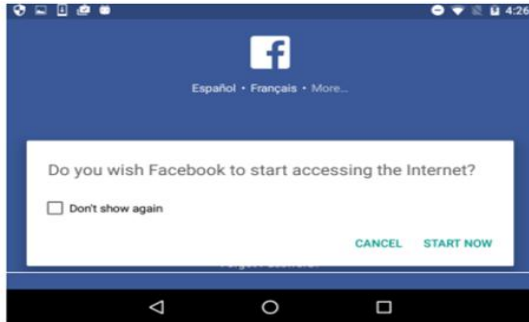
(b)



(c)

Screenshot Capturing Attack

- Objective: Capture screenshots of sensitive information.
- Steps:
 1. AppM monitors Facebook login activity.
 2. AppM captures screenshots of the login screen.
 3. AppM sends screenshots to a remote server.
- Visual: Example of captured screenshots.



(a)



(b)



(c)

Attack Performance

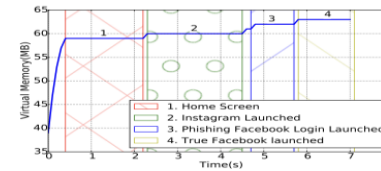
Time Cost:

- UI Phishing: 7.1 seconds.
- Activity-in-the-Middle: 5.8 seconds.
- Gallery Stealing: 4.6 seconds.
- Screenshot Capturing: 38.1 seconds.

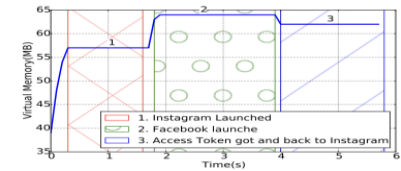
Memory Usage:

- Most attacks use less than 70MB.
- Gallery Stealing uses around 100MB.

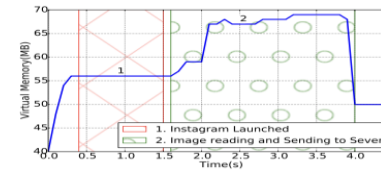
Attack	Time Cost (s)
UI Phishing	7.1
Man-in-the-Middle	5.8
Gallery Stealing	4.6
Screenshot Capturing	38.1



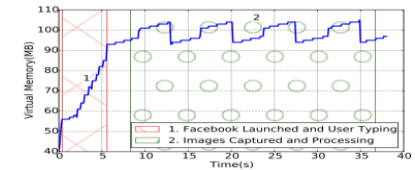
(a) UI Phishing



(b) Activity-in-the-middle



(c) Gallery Stealing



(d) Screenshot Capturing

Fig. 6: Memory Distribution Curves

Solution Overview

Task Interference Checking (TICK):

- Detects potential task interference.
- Warns users before installing risky apps.

Design Suggestions:

- Treat task mechanism as authorization.
- Avoid name conflicts in taskAffinity.

Task Interference Checking (TICK)

Components:

- Protected App Signature Database: Stores app conditions.
- Task Interference Table: Rules for task interference.
- Pre-processing: Extracts app metadata.
- Interference Checking: Detects risky apps.

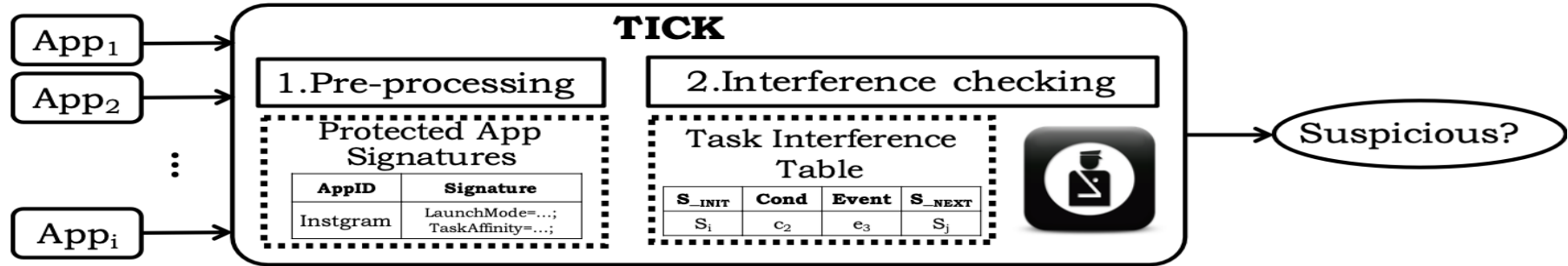


Fig. 7: Task Interference Checking Architecture

TICK in Action

Example:

- Instagram (AppB) and UI-Phishing app (AppM).
- TICK detects AppM's interference with Instagram.
- Warns user about potential risk.

Effectiveness: Scans 65 apps in 4-5 seconds.

Design Suggestions

Task as Authorization:

- Explicit ownership and authenticity of tasks.
- Avoid name conflicts in taskAffinity.

Security Mechanism:

- Similar to UNIX group mechanism.
- Prevent unexpected privilege escalation.

Evaluation

Effectiveness of TICK:

- Successfully detects UI-Phishing app.
- Low overhead (4-5 seconds for 65 apps).

Attack Feasibility:

- Lightweight attacks with minimal permissions.
- Low memory and battery usage.

Related Work

IPC Security:

- Android task hijacking (Ren et al.).
- iOS app isolation vulnerabilities (Xing et al.).

GUI Security:

- UI redressing attacks (Niemi et al.).
- UI inference attacks (Chen et al.).

Defense Mechanisms:

- AppFence (Hornyack et al.).
- WindowGuard (Ren et al.).

Conclusion

Key Findings:

- Android task mechanism can leak privileges.
- Four new active attacks demonstrated.
- TICK effectively detects task interference.

Future Work:

- Enhance task mechanism security.
- Improve detection tools for real-time protection.

Questions?
Thanks!